Programming Assignment – Part II

MENG544/ROBO554 Robot Mechanics: Kinematics, Dynamics, and Control

Dr. George Kontoudis
Department of Mechanical Engineering
Colorado School of Mines

Fall 2026

Overview and Expectations

Introduction

The purpose of this programming assignment is to create a core set of MATLAB functions and models that implement the main topics covered in the course. Although the programming assignment will span the full semester, it will be split into separately graded deliverables. Each deliverable depends on the successful completion of the prior deliverables, so it is important that you correct any errors that may occur. This is a daunting task that will require significant effort throughout the semester.

Plagiarism and Code Copying

Copying code is strictly forbidden. You may and should work together to specify the overall algorithm to be coded or to help with debugging, but the codding itself must be your own. All submissions must reflect the individual's effort to develop them. The learning from this assignment is by doing, debugging, and making it work. Copying does not help you learn. Thus, direct copying of functions, reports, and the like will be penalized strictly per university policy. The first offense results in an F for the entire course, the second offense results in an F for the course and a 1 year suspension, while the third offense results in expulsion.

Coding Style

This programming assignment will consist of writing MATLAB functions that are intended to be useful in the remaining of this course, future courses, and research. Thus, the style of these functions is critical for future usability. To enforce good style practices, part of the grade assigned for each function will be based on following good coding style and readability. Not only does this facilitate easy grading and future usability, but also helps with debugging and reduces the number of coding errors made.

For style guidelines see: https://www.mathworks.com/matlabcentral/fileexchange/46056-matlab-style-guidelines-2-0 (enter your Mines email at Mathworks webpage and login with your credentials to get access). At a minimum, the script file shall start with a code block that describes what the function does and shall be formatted as in the following example:

```
%  myFunctionName: Short description of what this function does.
%
% [output1, output2] = myFunctionName(input1, input2)
% A multiline description of the function would be appropriate.
%
% output1 = description of the first output (include units if appropriate)
% output2 = description of the second output (include units if appropriate)
%
% input1 = description of the first input (include units if appropriate)
% input2 = description of the second input (include units if appropriate)
%
% Your Name
% Your Student Number
% Course Number
% Date
function [output1, output2] = myFunctionName(input1, input2)
% YOUR FUNCTION CODE GOES HERE
end
```

Grading

The programming assignment is worth 20% of your total grade. Each function written will be graded on a three-point scale. One point will be awarded for style and readability, one point for effort, and one point for correctness.

- Upload all functions your code depends on that you write. For example in the second submission you must include your first submission. If your prior submissions had errors, you need to fix them or else the undesired functionality will propagate through the full semester.
- Use the function names I specify for each deliverable, or they will not be graded.

Time Management and Debugging

Together these programming assignments are 20% of the total course grade. As such, they will take time to complete. As the course progresses, you'll gain the information necessary to implement functions sequentially. **It is highly recommended that you work on the programming assignments in parallel with the weekly homework assignments.**

Coding based assignments are both nice and frustrating. Nice in that known test cases provided or generated by you can be used to verify if the implementation is correct. Frustrating in that either it works or it does not. I suggest that for every function you write you verify its correctness against a few known test cases. For example, if one function computes the forward transformation and another is its inverse transformation, verify that the inverse of the function of a value is the original value.

Function Testing

A testing script will be provided. It contains representative test cases for each function. You are expected to use the test script to assist the debugging and verification of your functions prior to submission.

Deliverable 2: Inverse Kinematics & Trajectory Planning

The following functions shall be submitted in a single zip folder uploaded to Canvas. A test-script will be provided along with project on Canvas. Final grading will be similar to the testing script, but with slightly different test cases. If you cannot pass the testing script, you will not do well in the grading.

Function Call	Description
`L = createLink(a, d, alpha, theta, offset, centOfMass, mass, inertia)`	Creates a structure with the following members: `a` - DH parameter a (meters) `d` - DH parameter d (meters) `alpha` - DH parameter α (radians) `theta` - joint angle provided by encoder (radians) `offset` - The number of radians (or meters for prismatic) different between the encoder orientation and the DH zero-angle, $\theta_{\text{DH}} = \Theta - \Theta_{\text{offset}}$ (equiv. equation for prismatic) `centOfMass` - the position of the link's center of mass `mass` - link mass (kg) `inertia` - link mass moment of inertia ($\text{kg}\,\text{m}^2$) isRotary - 1 if the joint is rotational, 0 if it is prismatic, and -1 if static All vectors and tensors are to be expressed in the link's coordinate frame. On call pass an empty array ([]) to the joint variable that changes, i.e., for a rotary joint a call would be: createLink(0.14, 3.4, pi/3, [], ...).
`H = dhFwdKine(linkList, paramList)`	Returns the forward kinematics of a manipulator with the provided DH parameter set. `linkList` - array of links, each created by `createLink` `paramList` - array containing the current state of joint variables, according to the robot's encoders
`[th1, th2, th3, th4, th5, th6, reachable] = abbInvKine (T_des, th_last)`	Returns the joint angles required to reach the desired transformation for the ABB arm. Make sure you handle joint limits, the rotation degeneracy, and unreachable states gracefully. `th1-th6` - 6 joint angles, according to the robot's encoders. These are real scalar values if `th_last` is provided, and are 8x1 vectors if `th_last` is not provided (one for each possible solution). `reachable` - true if the transform can be achieved, false otherwise `T_des` - desired homogeneous transform `th_last` - 6×1 vector of the last set of thetas used to select a specific solution. If `th_last` is not provided (check with exist ('`th_last`','`var`')), then provide all possibilities, otherwise return the closest solution to `th_last`. These should all be real values!

<table>
<tr><td></td><td>We have measured the parameters to be:
<table>
<tr><th>a [m]</th><th>d [m]</th><th>α [rad]</th><th>Θ_{offset} [rad]</th></tr>
<tr><td>0</td><td>0.290</td><td>$-\pi/2$</td><td>0</td></tr>
<tr><td>0.270</td><td>0</td><td>0</td><td>$\pi/2$</td></tr>
<tr><td>0.070</td><td>0</td><td>$-\pi/2$</td><td>0</td></tr>
<tr><td>0</td><td>0.302</td><td>$\pi/2$</td><td>0</td></tr>
<tr><td>0</td><td>0</td><td>$-\pi/2$</td><td>0</td></tr>
<tr><td>0</td><td>0.072</td><td>0</td><td>0</td></tr>
</table>
The encoders are shifted relative to the DH zero-angle convention. To convert use the following equation: $\Theta_{\text{arm}} = \theta_{\text{DH}} + \Theta_{\text{offset}}$</td></tr>
<tr><td><code>[p,v,a] = constAccelInterp(t, trajectory, transPercent)</code></td><td>Provides the position <code>p</code>, velocity <code>v</code>, and acceleration <code>a</code> at time t for a trajectory interpolated using the constant acceleration approach. Each of these are length M vectors.
<code>t</code> - time of interest (these may not be evenly spaced). If t is before or after the trajectory the function returns the same value as the initial or final time would, respectively.
<code>trajectory</code> - $N \times (M+1)$ array of points. There are N waypoints in the trajectory of dimension M. The first column is the time for each point in the trajectory and the remaining M columns are the point to be reached at that time.
<code>transPercent</code> - percentage of the trajectory time to use for the constant acceleration transition. This must be in the range [0, 0.5]. It is to be taken relative to the shorter duration segment when interpolating between two linear segments.</td></tr>
</table>